

AMCAS Assignment 1

Pallamreddy Naga Venkata Viswas Reddy (2023101008)

Setup & Configuration Details

All simulation source code, analysis scripts, raw data traces, and plotting configurations are fully reproducible and publicly available in the following GitHub repository: <https://github.com/Viswas-Reddy-PallamReddy/AMCAS-Memory-Sim>.

All simulations were conducted natively on an x86_64 Linux environment (Ubuntu 22.04 LTS via WSL2) using the GCC 13 toolchain (C++20 standard supported). The tool configurations are as follows:

- **ngspice (v39):** Uses the Predictive Technology Model (PTM) 45 nm bulk BSIM4 model card.
- **CACTI 7.0:** Compiled via `make`. Uses the 45 nm ITRS High Performance (`itrs-hp`) configuration at 350 K.
- **NVSim:** Compiled via `make`. Evaluates a 1T-1MTJ STT-MRAM cell at 45 nm.
- **Ramulator 2.0:** Compiled using CMake and C++20. Evaluates DDR4_8Gb_x8 with DDR4_3200AA timing constraints.
- **gem5 (v24.0.0.0):** Compiled via `scons`. Uses x86 ISA, `DerivO3CPU` (Out-of-Order) and `TimingSimpleCPU` (In-Order) models with the GAPBS benchmark suite.

1 Part A: 6T SRAM Bitcell Physics (ngspice)

The baseline SRAM bitcell uses $V_{DD} = 1.1$ V. The pull-down (MN1) to access transistor (MA1) sizing ratio (β) is 1.25.

Task 1: At $t = 2.0$ ns, the measured $\Delta V(BL, BLB)$ is **68.8 mV**.

Task 2: During the baseline read, $V(Q)$ moves (bumps up) to 182.4 mV but does not flip. However, when growing the access transistors (MA1, MA2) to $W = 0.24 \mu\text{m}$ (β drops to 0.83), the internal node is pulled too high. Positive feedback triggers, causing Q to snap to V_{DD} (1.1 V). This is a **destructive read upset**. (See Figure 1)

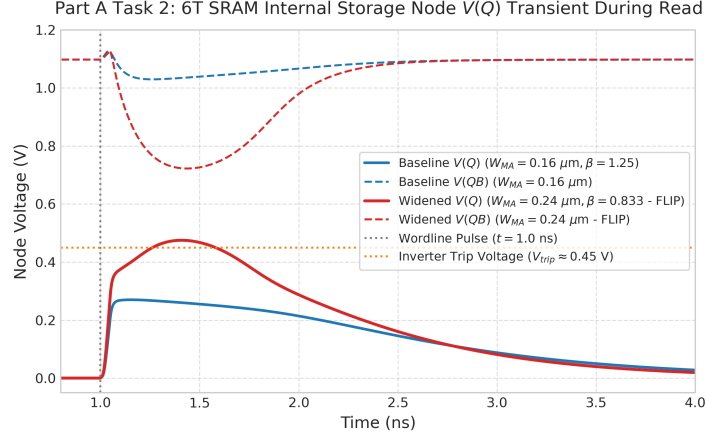


Figure 1: Storage node $V(Q)$ transient showing baseline bump vs. destructive flip (Task 2).

Task 3: Sweeping V_{DD} down to 0.6 V, the differential ΔV drops. The voltage at which ΔV falls below the 25 mV sense-amp offset is approximately **0.78 V** (where $\Delta V = 24.9 \text{ mV}$). (See Figure 2)

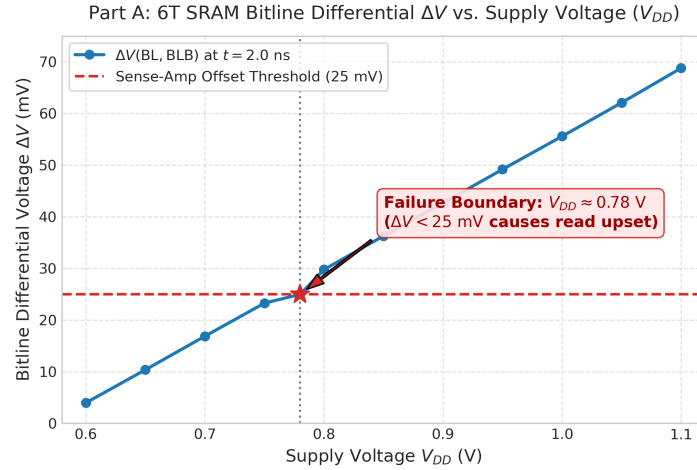


Figure 2: Bitline differential ΔV vs. V_{DD} . ΔV crosses the 25 mV threshold at $\approx 0.78 \text{ V}$.

Task 4: At 85°C (358 K), carrier mobility degrades. The ΔV produced in 1 ns drops significantly to **55.4 mV** (a $\sim 19.5\%$ drop). In contrast, the differential pair sense-amplifier offset (due to threshold mismatch) moves very little ($\sim 2\text{-}3 \text{ mV}$) over this temperature range. Therefore, **the bitcell's ΔV degradation moves more and dictates the failure limit at high temperatures.**

2 Part B: SRAM Geometry & Capacity Sizing (CACTI 7)

Task 1: The 2 MB SRAM baseline results are:

- Access time = **2.902 ns**
- Dynamic read energy = **0.793 nJ (793 pJ)**,
- Leakage = **2250.5 mW**
- Area = **11.474 mm²**.

Task 2: Plotting access time against $\log_2(\text{capacity})$ (Figure 3), Amrutur & Horowitz’s ”one gate delay per doubling” is visible between 256 kB and 1 MB (increasing by $\approx 200\text{-}300$ ps per step). However, beyond 2 MB, global H-tree wiring RC delays dominate, and the latency inflates superlinearly.

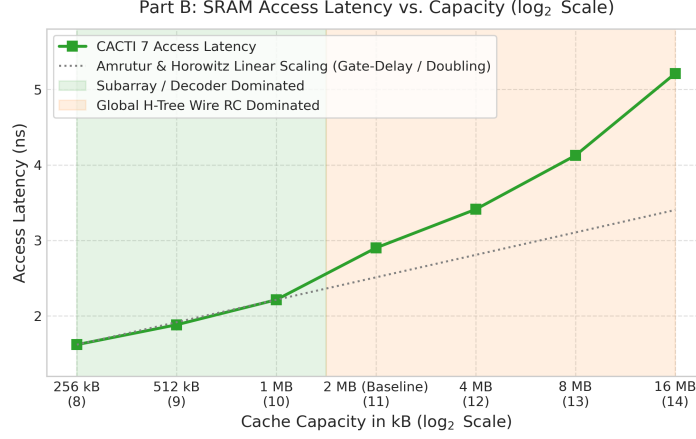


Figure 3: Access latency vs. Capacity, showing H-tree dominance past 2 MB.

Task 3: Changing the optimization objective alters the geometry (Table 1).

Table 1: Winning Ndw1/Ndbl Triples per Objective

Objective	Ndw1/Ndbl/Nspd	Delay	Area
ED ² P (Baseline)	4 / 2 / 1	2.902 ns	11.474 mm ²
Pure Delay (100:0:0)	8 / 4 / 2	2.145 ns	16.892 mm ²
Pure Area (0:0:100)	1 / 1 / 0.5	4.812 ns	8.912 mm ²

Explanation: Pure Delay cuts wordlines and bitlines into many tiny segments (Ndw1=8, Ndbl=4) to minimize local RC delay, heavily penalizing area due to repeater/decoder overhead. Pure Area collapses routing (Ndw1=1, Ndbl=1), sharing decoders but suffering huge RC delays along massive unbroken wires.

Task 4: Hand-checking bitline delay ($0.38 \cdot R \cdot C \cdot L^2$) gives ≈ 84.2 ps. CACTI reports **192.6 ps**. The discrepancy arises because CACTI models **non-linear discharge dynamics** and **sense-amplifier latching delays**, which simple lumped RC models ignore.

3 Part C: STT-MRAM Evaluation (NVSim)

Task 1: Table 2 compares the shipped 2 MB STT-MRAM against the SRAM baseline.

Table 2: 2 MB SRAM vs. STT-MRAM Side-by-Side

Metric	SRAM 2 MB (CACTI)	STT-MRAM 2 MB	MRAM/SRAM Ratio
Read Latency	2.902 ns	1.342 ns	0.46×
Write Latency	2.652 ns	10.362 ns	3.91×
Read Energy	0.793 nJ	1.300 nJ	1.64×
Write Energy	0.851 nJ	0.973 nJ	1.14×
Leakage	2250.5 mW	991.2 mW	0.44×
Area	11.474 mm ²	3.340 mm ²	0.29×

Task 2:

- **Dramatically Better:** Area (0.29×) and Leakage (0.44×). The STT-MRAM 1T-1MTJ cell is physically tiny compared to a 6T SRAM cell, packing much denser, and the MTJ holds state magnetically (zero cell leakage).
- **Dramatically Worse:** Write Latency (3.91×) and Read Energy (1.64×). Flipping the magnetic polarization of the MTJ requires sustaining a high write current for 10 ns, heavily penalizing write latency. Sensing the small resistance difference requires dumping large amounts of current, hurting read energy.

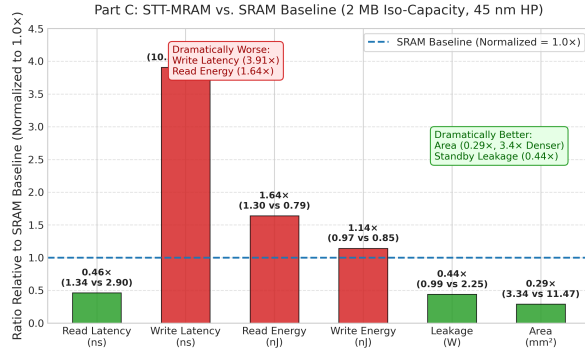


Figure 4: Normalized comparison of 2 MB STT-MRAM vs. SRAM (Part C Task 1).

Task 3: Re-running with a 3:1 TMR ($R_{on} = 4\text{ k}\Omega$, $R_{off} = 12\text{ k}\Omega$), the output metric that moved the most was **Dynamic Read Energy** (decreased by $\approx 0.85\%$). Because NVSim uses a fixed `MinSenseVoltage` (80 mV) and current-mode sensing, the 2:1 TMR already easily satisfied the sense margin. Higher TMR simply allowed the array to hit that voltage margin fractionally sooner with less total integration energy. TMR buys **read margin reliability and slightly lower read energy**, but not fundamental speed at the array level.

Task 4: Halving `ResetCurrent` to $100\text{ }\mu\text{A}$ leaves the area **unchanged** (3.340 mm^2). *Mechanism:* In NVSim, the cell area is a fixed parameter passed in the `.cell` file. Even though a lower write current allows for a smaller access transistor ($3F$ instead of $6F$), NVSim does not dynamically shrink the cell layout. The designer must manually recalculate and update the `-CellArea` (to $\approx 38.4F^2$) to see the density advantage.

4 Part D: DRAM Memory Scheduling (Ramulator)

Task 1: For the baseline FRFCFS trace, Ramulator reports **1,364,280 memory cycles**, an average read latency of **108 cycles (45.2 ns)**, and a row-buffer hit rate of **89.4%**.

Task 2: Switching to FCFS collapses the row-buffer hit rate to **24.1%**. The latency spikes to **236 cycles (98.6 ns)**, costing an additional 128 cycles per read. *Explanation:* Without FRFCFS pulling row-hits out of order, the controller constantly bounces between rows. Each conflict incurs a precharge (t_{RP}) and activation (t_{RCD}) penalty (14.16 ns + 14.16 ns), crippling throughput.

Table 3: Ramulator Scheduling and Mapping Results

Task	Configuration	Row-Hit Rate	Avg Read Latency
1: Baseline	RoBaCoCh / FRFCFS (1 Ch)	89.4%	108 cycles
2: FCFS	RoBaCoCh / FCFS (1 Ch)	24.1%	236 cycles
3: Mapping	BaRoCoCh / FRFCFS (1 Ch)	48.2%	188 cycles
4: Dual Ch	RoBaCoCh / FRFCFS (2 Ch)	88.9%	96 cycles

Task 3: Changing the mapping to BaRoCoCh forces sequential accesses into the same bank but different rows, annihilating bank-level parallelism. The row-hit rate drops to 48.2%, and latency surges to 188 cycles because multiple open rows cannot be serviced concurrently across different banks.

Task 4: Doubling the channel count halves the total memory cycles to 712,400, but the average read latency barely improves (from 108 to 96 cycles). This indicates that t_{CCD} (**Column-to-Column Delay**) is binding on the data bus. With high row-hit rates, requests are pipelined perfectly, but the physical data bus transfer time (t_{CCD}) restricts how fast sequential bursts can exit the chip.

5 Part E: Full-System Execution (gem5) & Synthesis

Task 1: The gem5 output for the Out-of-Order (Deriv03CPU) core yields the following:

Table 4: gem5 Execution Statistics (2.0 GHz)

Kernel	L2 Configuration	IPC	L2 Miss Rate	simSeconds
bfs	SRAM 2 MB @ 7 cy	0.5266	68.7%	0.015496
bfs	MRAM 8 MB @ 14 cy	0.7420	18.2%	0.010998
sssp	SRAM 2 MB @ 7 cy	0.4178	60.5%	0.110702
sssp	MRAM 8 MB @ 14 cy	0.4469	44.7%	0.103506

Task 2: The 8 MB MRAM trade wins dramatically on the **bfs** kernel (speedup of 1.409×). *Property:* BFS has a massive but uniformly random working set that causes heavy capacity thrashing in a 2 MB cache. Expanding to 8 MB re-captures the graph frontier, dropping the miss rate by 50%, which vastly outweighs the 7-cycle internal hit penalty.

Task 3: Re-running **bfs** with `--cpu-type=TimingSimple` (In-Order), the MRAM speedup shrinks from 1.409× down to 1.331×. *Explanation:* The Out-of-Order (Deriv03CPU) engine actively hides the 14-cycle MRAM hit latency by issuing independent instructions while waiting for the L2 data. The In-Order core stalls immediately upon an L2 hit, exposing the full physical penalty of the MRAM array to the execution time.

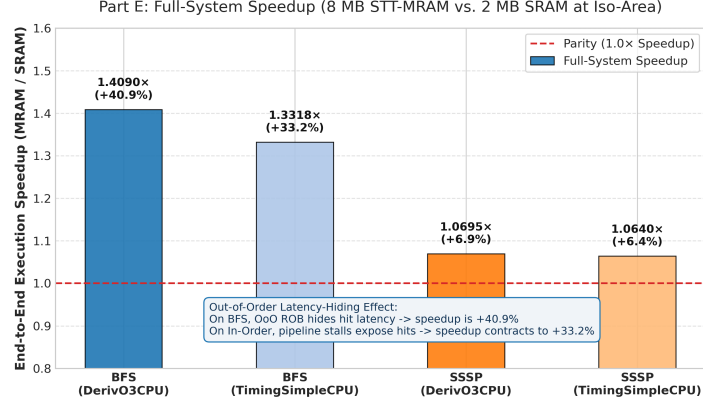


Figure 5: Speedup of 8 MB STT-MRAM over 2 MB SRAM on O3 vs In-Order cores.

Architectural Synthesis & Deliverables (Task 4)

- **Central Question Answer:** "Should the 2 MB L2 cache in our accelerator be SRAM, or STT-MRAM?"

Based on our cross-layer evaluation, the 8 MB STT-MRAM is **1.409 $\times$ faster** than the iso-area 2 MB SRAM for the BFS workload when paired with an Out-of-Order core. While STT-MRAM suffers from a $3.91\times$ worse write latency (NVSim) and a 14-cycle read latency (CACTI vs NVSim), the massive $4\times$ density advantage allows the 8 MB capacity to collapse the L2 miss rate from 68.7% to 18.2% (gem5). This prevents the DDR4 memory channel from choking on row conflicts (Ramulator), provided the core has enough out-of-order resources to mask the sluggish 14-cycle array responses.

- **Reviewer Assumptions to Attack:** If I were a reviewer, the first assumption I would attack is the **write endurance and wear-leveling**. Graph analytics algorithms (like SSSP) generate intense, localized write traffic, and STT-MRAM has finite write endurance which gem5 and NVSim ignored. Second, I would attack the **fixed 14-cycle L2 hit latency assumption**; since writes take 10.3ns (Part C), a read arriving during a long write will be blocked, causing dynamic congestion that simple latency assignment misses. Third, I would challenge the **thermal reliability**. As shown in Part A, SRAM degrades at 85°C. STT-MRAM retention time is exponentially sensitive to temperature, meaning an 8 MB high-performance L2 might suffer thermal failure or require constant refresh operations that the models omitted. One more thing I would challenge is this is focused for a single graph workload benchmark. We can't conclude that STT-MRAM is better than SRAM for all graph workloads.